

Linux file types

Regular File (Text/Binary)



- Regular files contain data, which can be either plain text, binary code, or executable programs. These files are the most common type on Linux systems. You can create, modify, and read them with various utilities.
- For example, text files can be read using commands like `cat`, and edited with text editors like `vim` or `nano`. There are no strict naming conventions for file extensions in Linux, and many system files have no extension at all.
- Regular files are represented by a single dash (-) at the beginning of file permissions when viewed with `ls -l`.

Directory



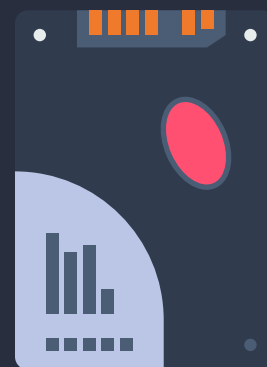
- A directory in Linux serves as a container for other files or directories, forming a hierarchical structure. Directories can be navigated using commands like `cd` and listed with `ls`.
- They play a critical role in organizing the filesystem, allowing users to store and access files efficiently. For example, `/home` holds user-specific files, while `/var/log` stores system log files.
- Directories are represented by a `d` at the beginning of their file permissions when viewed with `ls -l`.

Symbolic Link (Symlink)



- A symbolic link, or symlink, acts as a pointer to another file or directory, similar to a shortcut.
- This allows access to the target file without duplicating its contents. Symlinks are useful for reducing redundancy and creating flexible filesystem paths. The command `ln -s` is used to create a symlink.
 - When viewed using `ls -l`, symlinks are represented by an `l` and show the path they point to.

Block Device



- Block devices represent hardware that reads and writes data in fixed-size blocks, such as hard drives or SSDs.
- These devices interface with the Linux kernel to handle disk I/O operations. Common block devices include hard drives (`/dev/sda`) and NVMe devices (`/dev/nvme0n1`).
- Block devices are essential for mounting filesystems and managing disk storage.
- They are represented by a `b` in the file permissions listing.

Character Device



- Character devices handle data streams on a per-character basis and are typically used for devices like terminals or keyboards.
- They interact with the system one character at a time, in contrast to block devices that transfer data in larger chunks.
- Common character devices include `/dev/tty` for terminals and `/dev/null` for a special device that discards all input data.
- These devices are represented by a `c` in the file permissions

Named Pipe (FIFO)



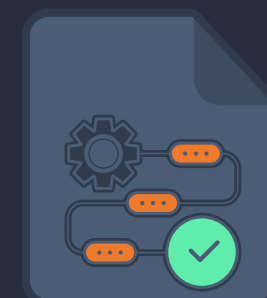
- Named pipes, or FIFOs, provide a method for inter-process communication, allowing one process to write data while another reads it in sequence.
- Unlike standard files, a FIFO doesn't store data on disk; instead, it facilitates real-time data exchange between processes.
- FIFOs can be created with the `mkfifo` command, and they appear in file listings with a `p` symbol at the start of their file permissions.

Socket



- Sockets enable communication between processes, both on the same machine and across a network. They play a critical role in network programming and inter-process communication.
- In Linux, sockets can be used for connecting to services or enabling communication between distributed systems. The command `ss` or `netstat` lists active sockets, and they are often found in `/var/run` directories.
- Sockets are represented by an `s` in file permissions listings.

Special Files



- Special files include unique system files like those found in the `/proc` and `/sys` directories.
- These files represent kernel and system information in a dynamic, virtual filesystem. For example, `/proc/cpuinfo` provides details about the system's CPU, and `/sys` contains kernel settings and parameters.
- These files are dynamically generated by the system and typically do not have their own distinct file type symbol.